Cours : Introduction à la Programmation Orientée Objet (POO) avec C#

La Programmation Orientée Objet (POO) est un paradigme de programmation qui repose sur la modélisation d'objets du monde réel et leurs interactions. Ce paradigme permet de mieux organiser et structurer le code, surtout pour des projets complexes.

Par Robert DIASSÉ

1. Pourquoi la POO ?

Exemple de la vie courante :

Imaginons une **agence automobile** qui gère plusieurs types de véhicules : voitures, motos, camions. Chaque véhicule a des propriétés (couleur, marque, vitesse) et des actions possibles (accélérer, freiner).

Si vous deviez gérer cela en programmation classique (structurée), vous auriez besoin de plusieurs variables et fonctions sans véritable organisation, ce qui deviendrait vite complexe.

La **POO** permet de modéliser chaque **type de véhicule** en tant qu'**objet** ayant ses propres propriétés et méthodes. Cela rend le code plus simple à comprendre, à maintenir et à étendre.

2. Concepts Fondamentaux de la POO

Concept	Description	Exemple dans la vie courante
Classe	Modèle ou plan permettant de créer des objets	Une recette de gâteau
Objet	Instance concrète d'une classe	Un gâteau spécifique préparé avec la recette
Encapsulation	Masquage des détails internes d'un objet pour protéger ses données	La télécommande cache ses circuits internes
Héritage	Capacité d'une classe à hériter des propriétés et méthodes d'une autre classe	Un vélo électrique hérite des propriétés de base d'un vélo
Polymorphisme	Capacité d'une méthode à se comporter différemment selon le contexte	Le bouton de démarrage d'une voiture peut démarrer différents moteurs

Concept	Description	Exemple dans la vie courante
Abstraction	Masquer les détails complexes pour ne montrer que l'essentiel	L'utilisation d'un four sans connaître son fonctionnement interne

3. Création d'une Classe et d'Objets en C#

Déclaration d'une Classe

Voici comment définir une classe en C#.

```
public class Voiture
{
    // Propriétés
    public string Marque;
    public string Couleur;
    public int Vitesse;

    // Méthode
    public void Demarrer()
    {
        Console.WriteLine($"{Marque} démarre.");
    }
}
```

Création d'un Objet

```
class Program
{
    static void Main(string[] args)
    {
        // Création d'un objet de type Voiture
        Voiture maVoiture = new Voiture();
        maVoiture.Marque = "Toyota";
        maVoiture.Couleur = "Rouge";
        maVoiture.Vitesse = 120;

        // Appel de la méthode
        maVoiture.Demarrer();
    }
}
```

Sortie :

```
Toyota démarre.
```

4. Encapsulation

L'encapsulation consiste à protéger les données en rendant les propriétés privées et en fournissant des méthodes publiques pour y accéder.

Exemple :

```
public class CompteBancaire
{
    private double solde;

    // Getter pour lire le solde
    public double GetSolde()
    {
        return solde;
    }

    // Setter pour déposer de l'argent
    public void Deposer(double montant)
    {
        if (montant > 0)
        {
            solde += montant;
        }
    }
}
```

Utilisation de la Classe

```
class Program
{
    static void Main(string[] args)
    {
        CompteBancaire compte = new CompteBancaire();
        compte.Deposer(100);
        Console.WriteLine($"Solde : {compte.GetSolde()} euros");
    }
}
```

Sortie :

```
Solde : 100 euros
```

5. Héritage

L'héritage permet de créer une classe qui hérite des propriétés et méthodes d'une autre classe.

Exemple :

```
public class Animal
{
    public void Manger()
    {
        Console.WriteLine("L'animal mange.");
    }
}

public class Chien : Animal
{
    public void Aboyer()
    {
        Console.WriteLine("Le chien aboie.");
    }
}
```

Utilisation

```
class Program
{
    static void Main(string[] args)
    {
        Chien monChien = new Chien();
        monChien.Manger(); // Héritée de Animal
        monChien.Aboyer(); // Propre à Chien
    }
}
```

Sortie :

```
L'animal mange.
Le chien aboie.
```

---

6. Polymorphisme

Le polymorphisme permet de redéfinir une méthode dans une classe dérivée.

Exemple :

```
public class Animal
{
    public virtual void Communiquer()
    {
        Console.WriteLine("L'animal communique.");
    }
}

public class Chat : Animal
{
    public override void Communiquer()
    {
        Console.WriteLine("Le chat miaule.");
    }
}
```

Utilisation

```
class Program
{
    static void Main(string[] args)
    {
        Animal animal = new Chat();
        animal.Communiquer();
    }
}
```

Sortie :

```
Le chat miaule.
```

---

7. Exemple Pratique : Gestion d'une Bibliothèque

Créer une application simple pour gérer une bibliothèque avec des classes représentant des livres.

Solution :

```
using System;
using System.Collections.Generic;

public class Livre
{
    public string Titre { get; set; }
    public string Auteur { get; set; }
```

```
    public void AfficherDetails()
    {
        Console.WriteLine($"Titre : {Titre}, Auteur : {Auteur}");
    }
}

class Program
{
    static void Main(string[] args)
    {
        List<Livre> bibliotheque = new List<Livre>
        {
            new Livre { Titre = "1984", Auteur = "George Orwell" },
            new Livre { Titre = "Le Petit Prince", Auteur = "Antoine de Saint-
Exupéry" }
        };

        foreach (Livre livre in bibliotheque)
        {
            livre.AfficherDetails();
        }
    }
}
```

---

8. Exercices à Faire

1. **Créer une Classe Personne :**
   Créez une classe `Personne` avec des propriétés `Nom`, `Prenom` et une méthode `SePresenter()` qui affiche une présentation de la personne.

2. **Gestion de Compte Bancaire :**
   Modifiez la classe `CompteBancaire` pour permettre les retraits d'argent avec une vérification du solde.

3. **Simulation d'Animaux :**
   Créez une hiérarchie de classes pour différents animaux (`Animal`, `Chien`, `Chat`) et implémentez une méthode `Communiquer()` polymorphe.

4. **Application de Bibliothèque :**
   Étendez l'exemple de la bibliothèque pour permettre l'ajout de nouveaux livres via une interface utilisateur.

---

9. Concepts Clés à Retenir

- Une **classe** est un modèle pour créer des objets.
- Un **objet** est une instance concrète d'une classe.
- **Encapsulation** protège les données d'une classe.
- **Héritage** permet de réutiliser et d'étendre des fonctionnalités.
- **Polymorphisme** permet une flexibilité dans le comportement des méthodes.